



Sistema de Gestión para Restaurante

Diseño, implementación y migración de una base de datos en PostgreSQL

Bases de Datos · Grupo 1 · Semestre 2026-2

Profesor: Ing. Fernando Arreola Franco

Bojorquez C. · Gabriel H. · Martínez O. · Parra B. · Parra V. · Yáñez

Agenda

- 1 Planteamiento y objetivo
- 2 Plan de trabajo
- 3 Diseño de la base de datos
- 4 Implementación: DDL, funciones y triggers
- 5 Vistas e índices
- 6 Migración de datos (ETL)
- 7 Resultados
- 8 Conclusiones

Planteamiento del problema



Un restaurante necesita digitalizar su operación diaria.

Personal

Empleados (cocineros, meseros, administrativos), con varios puestos posibles, sus teléfonos y dependientes.

Productos

Platillos y bebidas con receta, precio, disponibilidad y categoría.

Órdenes

Folio, fecha/hora, mesero responsable, productos y totales por producto y por venta.

Facturación

Datos fiscales del cliente cuando solicita factura.

Objetivo y alcance



Construir una base de datos relacional en PostgreSQL que centralice la operación del restaurante, automatice los cálculos y validaciones del negocio, y permita migrar la información diaria a una segunda base de datos.

Modelo íntegro

Diseño normalizado con especializaciones y restricciones de integridad.

Automatización

Secuencias, funciones y triggers para folios, totales y validaciones.

Consultas

Funciones y vistas que responden a los requerimientos del negocio.

Migración (ETL)

Exportación a texto e importación orquestada a la BD destino.

Plan de trabajo

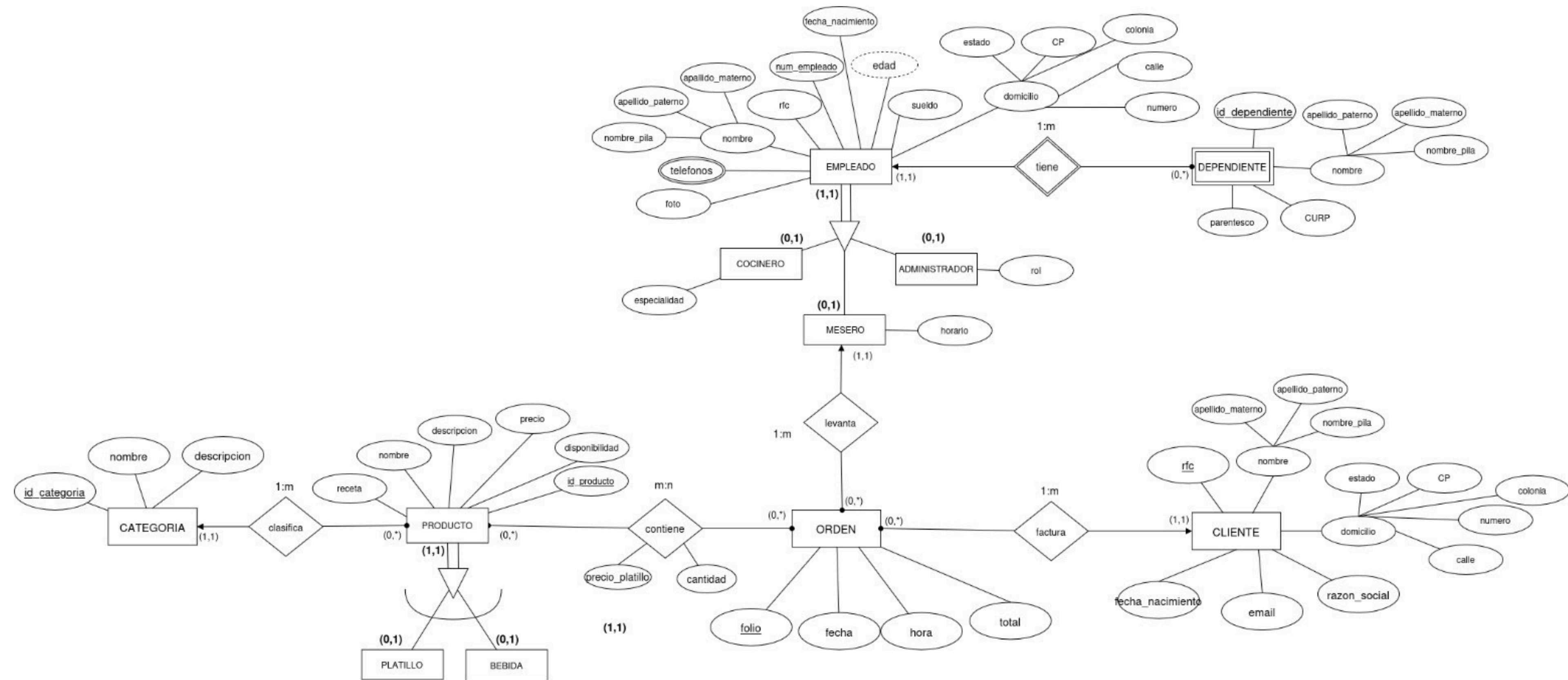
Fases del proyecto

- Análisis de requerimientos
- Diseño (MER / MR)
- Implementación de la BD
- Funciones de consulta y pruebas
- Migración ETL y orquestación
- Documentación

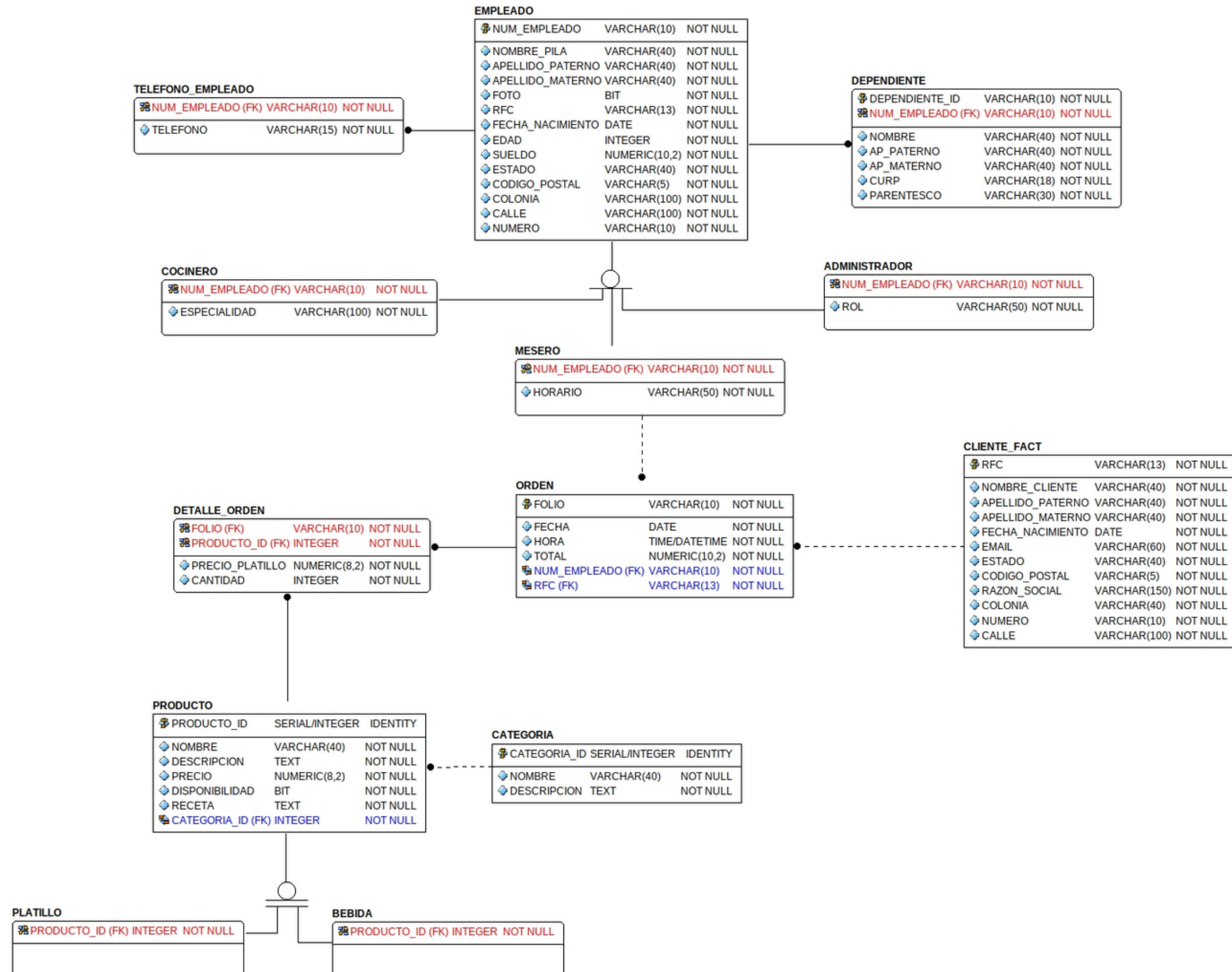
Responsabilidades

Parra Bello	Arquitectura y diseño de tablas
Parra Vera	Secuencias
Bojorquez Covarrubias	Funciones de consulta y triggers
Martínez Ortiz	Vistas e índices
Yañez y Gabriel	Orquestación con Pentaho

Diseño · Estrategia de modelado



Diseño · Estrategia de modelado



Implementación · Secuencias y DDL

- Secuencias para identificadores autoincrementales.
- DDL creado en orden de dependencia entre tablas.
- Integridad con FK + ON DELETE / ON UPDATE.
- Discriminador tipo y CHECK en PRODUCTO.

```
CREATE SEQUENCE folio_seq
  START WITH 1 INCREMENT BY 1
  MINVALUE 1 NO MAXVALUE NO CYCLE;

CREATE TABLE PRODUCTO (
  producto_id INTEGER DEFAULT
    nextval('producto_id_seq'),
  precio NUMERIC(8,2) CHECK (precio > 0),
  tipo VARCHAR(10) CHECK (tipo IN
    ('platillo','bebida')), ...
);
```


Función generar_folio()

 Genera el folio con formato ORD-XXX consumiendo la secuencia.

```
CREATE OR REPLACE FUNCTION generar_folio()
RETURNS VARCHAR(10) AS $$
DECLARE
    v_numero INTEGER;
    v_folio VARCHAR(10);
BEGIN
    v_numero := nextval('folio_seq');
    v_folio := 'ORD-' || LPAD(v_numero::TEXT, 3, '0');
    RETURN v_folio;
END;
$$ LANGUAGE plpgsql;
```

Salida

seq 1	ORD-001
seq 50	ORD-050
seq 100	ORD-100
seq 1000	ORD-1000

Asignado automáticamente por un trigger BEFORE INSERT en ORDEN.

Triggers · Automatización



Cuatro triggers mantienen folios, totales y validaciones consistentes.

#	Trigger	Tabla	Evento
1	trg_generar_folio	ORDEN	BEFORE INSERT
2	trg_insertar_detalle	DETALLE_ORDEN	BEFORE INSERT
3	trg_actualizar_detalle	DETALLE_ORDEN	BEFORE UPDATE
4	trg_eliminar_detalle	DETALLE_ORDEN	AFTER DELETE

El #4 es AFTER DELETE: el total solo se recalcula bien una vez borrada la fila.

Trigger · Generación de Folio de Orden

- No se escribe el folio manualmente
- Intercepta antes de insertar la fila
- Asigna el folio automáticamente desde la secuencia

```
CREATE OR REPLACE FUNCTION fn_generar_folio_orden()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.folio := generar_folio();  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_generar_folio  
    BEFORE INSERT ON ORDEN  
    FOR EACH ROW  
    EXECUTE FUNCTION fn_generar_folio_orden();
```

Trigger • Antes de inserción de detalle

- Valida que el producto exista y esté disponible.
- Calcula el subtotal por producto.
- Actualiza el total de la orden.

```
SELECT disponibilidad, precio, nombre
  INTO v_disponible, v_precio, v_nombre
  FROM PRODUCTO
  WHERE producto_id = NEW.producto_id;

  IF v_disponible = FALSE THEN
    RAISE EXCEPTION 'El producto "%" no esta
disponible actualmente.', v_nombre;
  END IF;

NEW.precio_platillo := NEW.cantidad * v_precio;

UPDATE ORDEN
  SET total = (
    SELECT COALESCE(SUM(precio_platillo), 0) +
NEW.precio_platillo
    FROM DETALLE_ORDEN
    WHERE folio = NEW.folio
  )
  WHERE folio = NEW.folio;

RETURN NEW;
```

Trigger · Antes de actualización de detalle

- Recalcula el subtotal
- Actualiza el total

```
NEW.precio_platillo := NEW.cantidad * v_precio;

UPDATE ORDEN
  SET total = (
    SELECT COALESCE(SUM(
      CASE
        WHEN folio = NEW.folio AND
producto_id = NEW.producto_id
        THEN NEW.precio_platillo
        ELSE precio_platillo
      END
    ), 0)
    FROM DETALLE_ORDEN
    WHERE folio = NEW.folio
  )
WHERE folio = NEW.folio;

RETURN NEW;
```


Trigger • Después de eliminación en detalle

- Recalcula el total de la orden sin el registro eliminado.

```
UPDATE ORDEN
  SET total = (
    SELECT COALESCE(SUM(precio_platillo), 0)
    FROM DETALLE_ORDEN
    WHERE folio = OLD.folio
       AND producto_id != OLD.producto_id
  )
WHERE folio = OLD.folio;

RETURN OLD;
```

Funciones de consulta

ordenes_mesero_hoy

```
DECLARE
    v_es_mesero BOOLEAN;
BEGIN
    SELECT EXISTS(SELECT 1 FROM mesero WHERE
num_empleado = p_num_empleado) INTO v_es_mesero;
    IF NOT v_es_mesero THEN
        RAISE EXCEPTION 'Error: El empleado con número
% no es un mesero válido.', p_num_empleado;
    END IF;
    RETURN QUERY
    SELECT
        COUNT(folio)::BIGINT,
        COALESCE(SUM(total), 0)::NUMERIC
    FROM orden
    WHERE num_empleado = p_num_empleado
        AND fecha = CURRENT_DATE;
END;
$$ LANGUAGE plpgsql;
```

- Recibe número de empleado
- Regresa cuántas órdenes levantó hoy y cuánto dinero sumaron.

Funciones de consulta



ventas_por_fechas

```
IF p_fecha_fin IS NULL THEN
    p_fecha_fin := p_fecha_inicio;
END IF;

RETURN QUERY
SELECT
    COUNT(folio)::BIGINT,
    COALESCE(SUM(total), 0.00)::NUMERIC
FROM orden
WHERE fecha >= p_fecha_inicio
    AND fecha <= p_fecha_fin;
END;
$$ LANGUAGE plpgsql;
```

- Recibe fecha o un rango
- Regresa cuántas órdenes hubo y cuánto se vendió en total.

Vistas



Tres vistas resuelven los requerimientos de consulta del negocio.

Platillo más vendido

Se usaron los atributos:

PRODUCTO.producto_id,
PRODUCTO.nombre,
PRODUCTO.descripcion,
PRODUCTO.precio

SUM(cantidad), JOIN PRODUCTO-
DETALLE_ORDEN, un WHERE para
indicar que solo queremos platillo,
GROUP BY, ORDER BY DESC, LIMIT 1.

Factura

Se usaron los atributos:

ORDEN.folio, ORDEN.fecha, ORDEN.hora,
ORDEN.total, CLIENTE_FACT.rfc,
CLIENTE_FACT.nombre_cliente,
CLIENTE_FACT.email,
CLIENTE_FACT.estado,
CLIENTE_FACT.codigo_postal,

3 JOINS (ORDEN, CLIENTE_FACT,
DETALLE_ORDEN, PRODUCTO); subtotal =
cantidad × precio; solo órdenes con RFC.

Productos no disponibles

Se usaron los atributos:

PRODUCTO.nombre

Nombre de productos WHERE
disponibilidad = FALSE.

Índices · Rendimiento



Tipo B-tree: ideal para igualdad, orden y rangos; es el predeterminado en PostgreSQL.

Aplicados sobre llaves foráneas y la columna fecha, para acelerar JOINS y las funciones de consulta por rango.

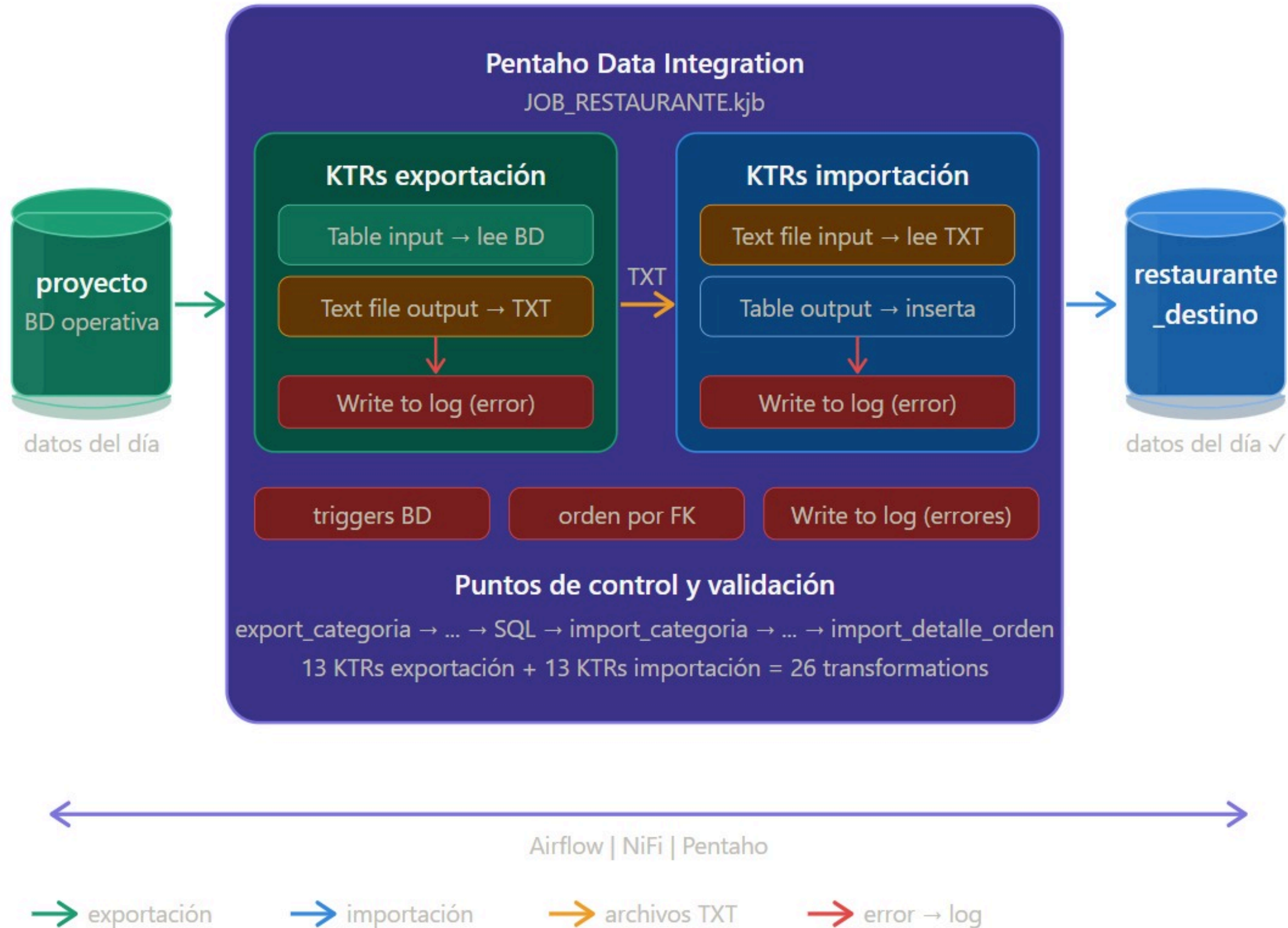
```
-- Para producto_id
CREATE INDEX idx_DetalleOrden_producto_id ON DETALLE_ORDEN(producto_id);

-- Para rfc
CREATE INDEX idx_Orden_rfc ON ORDEN(rfc);

-- Para num_empleado
CREATE INDEX idx_Orden_num_empleado ON ORDEN(num_empleado);

-- Para fecha
CREATE INDEX idx_orden_fecha ON ORDEN (fecha); -- ventas_por_fechas (rango)
```

Presentación · Migración de datos (ETL)



Resultados · Requerimientos cubiertos

- ✓ Folio con formato ORD-001 automático
- ✓ Totales por producto y por venta + validación de disponibilidad
- ✓ Órdenes y total de un mesero en el día
- ✓ Vista del platillo más vendido
- ✓ Vista tipo factura
- ✓ Productos no disponibles
- ✓ Ventas por fecha o por rango de fechas
- ✓ Índice justificado (B-tree)
- ✓ Migración orquestada origen → destino con manejo de errores

Conclusiones

Diseño primero

Un buen modelo de tablas y llaves evita datos basura y facilita todo lo demás.

Automatización real

Triggers y funciones hacen que la BD trabaje sola: folios, totales y validaciones.

Robustez

COALESCE y excepciones controlan nulos y errores; el sistema no colapsa ante datos malos.

Migración con criterio

La orquestación depende del orden de las llaves foráneas y de validar lo que entra.



Gracias

¿Preguntas?